

Introduction to LINQ

اولا هنفهم شوية حاجات ال LINQ معتمدة عليهم.

Expression vs Statement

1. Statement

A statement can consist of a single line of code that ends in a semicolon, or a series of single-line statements in a block.

A statement block is enclosed in {} bracts and can contain nested blocks.

أمثلة:

```
//Declaration statemens
int counter;

//Assignment statement
counter = 1;

//Initilization (Declare + Assign)
double pi = 3.14;

//For and foreach statement
for (int i = 0; i < 10; i++)
{

}

foreach (var item in new int[] {1,3,4})
{

}

// if - else
if (counter == 1)
{

}

// switch, jump, do, while
```

2. Expression

An expression is a sequence of operators and operands that yield a value.

هو أي جزء من الكود ينتج قيمة (Value)

أمثلة:

```
//Expression
var r = 10; //-> statement
var area = pi * (r * r); //-> Expression pi * (r * r) yield a value
var v = Math.Sqrt(25); //-> Expression Math.Sqrt(25) yield a value

// Method invocation (this method yield void)
Console.WriteLine(area);

//new object creation
var names = new string[] { "menna", "ahmed", "momen" };
```

العلاقة بينهم

Statement can include expression and not the other way.

مثال:

```
int result = 5 + 3;
```

`int result = 5 + 3;` → Statement

`5 + 3` → Expression

مثال كمان:

```
bool isAdult = age >= 18;
```

`bool isAdult = age >= 18;` → Statement

`age >= 18` → Expression

ملخص:

Statement	Expression
يُنْفِذ أمر	يُنْتِج قيمة
قد يحتوي على Expression	يمكن استخدامه داخل Statement

Pure Function vs Impure Function

1. Pure Function

1. نفس المدخلات → نفس المخرجات دائماً

يعني: `Add(2, 3)` كل مرة ترجع 5

2. لا تؤثر على أي شيء خارجها

يعني لا:

- تطبع على الشاشة
- تعدل متغير Global
- تقرأ ملف
- تكتب ملف
- تتعامل مع قاعدة بيانات

عادي اعدل على local variable بس لو عدلت على global تبقى impure

مش بتسبب Side Effects

عشان كد بتكون أسهل في الاختبار (Testing) وأقل في الأخطاء ومناسبة جداً لل Functional Programming

مثال:

```
static int Add(int a, int b) // دائما هترجع 5 ومش بتاثر ع حاجه ثانيه براها
{
    return a + b;
}
```

2. Impure Function

هي أي Function تكسر أحد الشرطين لل Pure.

```
static DateTime GetNow()
{
    return DateTime.Now;
}
```

لان كل مره هت return نتائج مختلفة.

تعتمد على External State وهو System Clock.

امثله على Pure Function, Impure Function :

```
internal class Program
{
    //Globel variable
    static List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    static void Main(string[] args)
    {
        Print(numbers);
        //AddInteger1(7);
        Print(numbers);
        int num = 5;
        //AddInteger2(ref num);
        //AddInteger3();
        var r = AddInteger4(numbers, num);
        Print(r);
    }
    static void Print(List<int> numbers)
    {
        foreach (var n in numbers)
        {
            Console.Write(n+" ");
        }
        Console.WriteLine();
    }
    static void AddInteger1(int num)
    {
        numbers.Add(num); // impure mutate globel variable.
    }
}
```

```

}

static void AddInteger2(ref int num)
{
    num++; // impure mutate parameter.
    numbers.Add(num);
}

static void AddInteger3()
{
    numbers.Add(new Random().Next()); // impure interaction with outside
world.

}

static List<int> AddInteger4(List<int> numbers , int num)
{
    var result = new List<int>(numbers); //global هنا انا مغيرتش في ال
list
    //pure function
    result.Add(num);
    return result;
}

}

```

Pure Function

Robust and testable code

Prefer declarative over imperative coding

Functional programming

Programing paradigm for developing software using function

Pure function is the key part of functional programming

عندنا اكثر من طريقه نقدر نمثل بيها ال function زي ال Methods - Delegates - Lambda expression

تعالى ناخذ برنامج ونشوف اي المشاكل اللي فيه ونبدأ نحلها

عندي مجموعة موظفين وببدا اعمل مجموعة استعلامات معينه عليهم

```

using System;
using System.Collections.Generic;
using System.Text;

namespace FunctionalProgramming
{
    public class ExtensionProcedural
    {
        public static IEnumerable<Employee>
GetEmployeesWithFirstNameStartsWith(string value)
        {
            var employees = Repository.LoadEmployees();
            foreach (var employee in employees)
            {
                if(employee.FirstName.ToLower().StartsWith(value.ToLower()))
                {
                    yield return employee;
                }
            }
        }

        public static IEnumerable<Employee>
GetEmployeesWithSalaryGreaterThan(decimal minSalary)
        {
            var employees = Repository.LoadEmployees();
            foreach (var employee in employees)
            {
                if (employee.Salary > minSalary)
                {
                    yield return employee;
                }
            }
        }

        public static IEnumerable<Employee> GetEmployeesByDepartment(string
deptName)
        {
            var employees = Repository.LoadEmployees();
            foreach (var employee in employees)
            {
                if (employee.Department.Equals(deptName,
StringComparison.OrdinalIgnoreCase))
                {
                    yield return employee;
                }
            }
        }
    }
}

```

```

    public static IEnumerable<Employee> GetEmployeesHiredAfter(DateTime
date)
    {
        var employees = Repository.LoadEmployees();
        foreach (var employee in employees)
        {
            if (employee.HireDate > date)
            {
                yield return employee;
            }
        }
    }

    public static IEnumerable<Employee>
GetEmployeesWithInsuranceAndNoPension()
    {
        var employees = Repository.LoadEmployees();
        foreach (var employee in employees)
        {
            if (employee.HasHealthInsurance && !employee.HasPensionPlan)
            {
                yield return employee;
            }
        }
    }

    public static void Print<T>(IEnumerable<T> source, string title)
    {
        if (source == null)
            return;
        Console.WriteLine();
        Console.WriteLine("-----");
-----");
        Console.WriteLine($"|{title.PadRight(52, ' ')}|");
        Console.WriteLine("-----");
-----");
        Console.WriteLine();
        foreach (var item in source)
            Console.WriteLine(item);
    }
}

```

المشاكل اللي عندي:

1. يعتمد على مكان ثاني ازاى

هنا `var employees = Repository.LoadEmployees();` هنا ال `ExtensionProcedural` يعتمد على ال `Repository`

يعني يعتمد على `external dependencies` طبعاً اتكلمنا في الجزء دا قبل كد في ال `Dependency injection` دا بيسبب مشاكل زي انه بيكون صعب في ال `test` وصعب في الصيانة ومفيش مرونه

2. Violation of Single Responsibility.

الميثود بتعمل:

- Load Data
- Filter
- Return

ثلاث مسؤوليات.

ولاكن المفروض تعمل

فقط Filtering.

3. ال `method` بتاعتي مش `pure`.

4. الكود بتاعي `imperative`.

أنا عايزين نعبر عنه بطريقة `pure` عشان نقلل المشاكل دي

هنعمل كد

```
using System;
using System.Collections.Generic;
using System.Text;

namespace FunctionalProgramming
{
    public static class ExtensionFunctional01
    {
        public static IEnumerable<Employee> Filter (IEnumerable<Employee>
source, Func<Employee, bool> predicate)
        {
            foreach (var employee in source)
            {
                if (predicate(employee))
                {
                    yield return employee;
                }
            }
        }
    }
}
```

```

    }
}
public static void Print<T>(IEnumerable<T> source, string title)
{
    if (source == null)
        return;
    Console.WriteLine();
    Console.WriteLine("-----");
    Console.WriteLine($"|{title.PadRight(52, ' ')}|");
    Console.WriteLine("-----");
    Console.WriteLine();
    foreach (var item in source)
        Console.WriteLine(item);
}
}
}

```

هنا انا استخدمت ال delegate بدل م اخلي ال method بتاعتي تشتغل ع condition معين لا هيا بتاخذ method او lambda expression انا بق ابعث ال condition اللي انا عايزه

تاني حاجه انا خليت ال method تستقبل الداتا من بره ودا حل من ضمن الحلول اللي بنقل الشاكل اللي هو **Method Parameter Injection**

كمان بتحقق ال Higher order function

Higher order function:

Function that takes Function as an argument or return function.

ال LINQ بق معظم ال functions اللي فيها هي Higher order function

كمان طالما انا دايم ابعث list من ال employees فمفيش ضرورة اني ابعثها كل مره ك parameter

بعمل كد ازاي بعلم ال parameter اللي هينادي ال method ب this keyword

Extension Method يعني اخليها

هنطبقها برضو في الكود

```

public static class ExtensionFunctional02
{
    public static IEnumerable<Employee> Filter(this IEnumerable<Employee>
source, Func<Employee, bool> predicate)
    {
        foreach (var employee in source)

```

```

        {
            if (predicate(employee))
            {
                yield return employee;
            }
        }
    }

    public static void Print<T>(this IEnumerable<T> source, string title)
    {
        if (source == null)
            return;
        Console.WriteLine();
        Console.WriteLine("-----");
        Console.WriteLine($"|{title.PadRight(52, ' ')}|");
        Console.WriteLine("-----");
        Console.WriteLine();
        foreach (var item in source)
            Console.WriteLine(item);
    }
}

```

يبق كد عرفنا ان ال LINQ بتعتمد على ال Expression و بتطبق ال (Pure fun) Functional programming

LINQ → Language Integrated Query لغة الاستعلام المتكامل

اي عمليات استعلام تكون موحدة عن طريق ال LINQ

زي مثلا نكلم الداتا بيز , استعلام على object موجود في Memory .. توحيد عمليات الاستعلام باستخدام ال LINQ

LINQ → Language Feature + Standard Query Operators

MS.NET Component (Language + Feature)

Type safe queries in code (type conflict مش هيحصل مشاكل زي)

Query local object IEnumerable → "Object in memory or built in objects"

LINQ to remote (SQL) IQueryable

Set of Query operator Extension method

يعني البرنامج اللي اشتغلنا عليه فوق هو اصلا دا شكل ال LINQ

Instead of creating our custom method

```
public static IEnumerable<Employee> Filter
    (this IEnumerable<Employee> source, Func<Employee, bool> predicate)
{
}
```

We can rely on LINQ

Where method

```
public static IEnumerable<TSource> Where<TSource>
    (this IEnumerable<TSource> source, Func<TSource, bool> predicate)
{
}
```

Write LINQ statements using

1. Method syntax

```
var result = numbers
    .Where(x => x % 2 == 0);
```

2. Query syntax

```
var result = from n in numbers
              where n % 2 == 0
              select n;
```

ال Query syntax بتترجم اصلا الى Method syntax.

اي فائدة ال LINQ

1. Familiar language.
2. Unified Syntax.
3. Less coding.
4. Type safety at compile time.
5. Deferred Execution.
6. IntelliSense Support.

7. **Declarative Coding.**
8. **Easy Unit Testing.**
9. **Integration with EF Core.**